

Funções De Alta Ordem

João Eduardo Montandon
DCC024

Uma função de alta ordem é uma função que recebe outras funções como parâmetro, ou produz uma como retorno.

Quais são as implicações disto? 🤔

Casamento De Padrões Pt 2

```
<case-exp> ::= case <expression> of <match>  
<match> ::= <rule> | <rule> '|' <match>  
<rule> ::= <pattern> => <expression>
```

- `<rule>` : uma expressão é executada se uma expressão se encaixar no padrão.
 - `<match>` : permite encadear diversas regras em um mesmo match.
 - `<case-exp>` : Exemplo de uso do `<match>`
-

```
- case 1 + 1 of  
= 3 => "three" |  
= 2 => "two" |  
= _ => "hmmm";  
val it = "two" : string
```

Soa familiar? 🤔

Diferencial em SML: os padrões também podem ser desmembrados via casamento de padrões.

```
case x of  
  _ :: _ :: c :: _ => c |  
  _ :: c :: _ => c |
```

```
c :: _ => c |  
nil => 0;
```

Como simular uma expressão condicional?

```
case exp_1 of  
  true => exp_1 |  
  false => exp_3;
```

Funções Como Valores

- Em linguagens imperativas, funções são atreladas a um único e permanente nome.
- Linguagens funcionais permitem atribuir funções a variáveis, tratando-as como qualquer outro valor.

A função `f` ➡ A função ligada ao nome `f`.

```
- val toInt = ord;  
val toInt = fn : char -> int  
- toInt #"a";  
val it = 97 : int
```

Em SML, alguns operadores na verdade são implementados como funções.

```
- ~;  
val it = fn : int -> int  
- val menos = ~;  
val menos = fn : int -> int  
- menos 2;  
val it = ~2 : int
```

O método `op` permite extrair a função de um determinado operador.

```
- op +;
val it = fn : int * int -> int
- val soma = op +;
val soma = fn : int * int -> int
- soma (4, 3);
val it = 7 : int
```

Funções Anônimas

- Também é possível definir funções sem atribuí-las a um nome.
- Essas funções são conhecidas como **funções anônimas**.

```
- fn x => x + 2;
val it = fn : int -> int
- it 2;
val it = 4 : int
```

Podemos computar a função anônima diretamente, caso seus parâmetros sejam fornecidos.

```
- (fn x => x + 2) 2;
val it = 4 : int
```

Soa familiar? 🤔

- Funções anônimas ampliam a expressividade do código.
 - Agora podemos parametrizar ações, ao invés de apenas dados.
 - Muito útil em alguns cenários.^[1]
-

Exemplo: Merge Sort

E se eu quiser escolher o modo de ordenação? 🤔

```
fun menorQue (a,b) = a < b;

- mergeSort ([100,1,40,~2,~25,14,4,234,6,10,0], menorQue);
val it = [~25,~2,0,1,4,6,10,14,40,100,234] : int list
```

```
- mergeSort ([100,1,40,~2,~25,14,4,234,6,10,0], op >);  
val it = [234,100,40,14,10,6,4,1,0,~2,~25] : int list
```

Exemplo completo ao [final do material](#).

Funções De Alta Ordem

Toda função tem uma **ordem**, definida a seguir:

1. **Ordem 1**: função que não recebe ou retorna uma função como resultado.
2. **Ordem n+1**: função possui **ordem n+1**, onde **n** é o valor da função de maior ordem recebida como parâmetro ou retornada.

Qualquer função que possua **ordem > 1** é chamada de **função de alta ordem**.

Qual a diferença entre as duas funções? 🤔

```
val f1 = fn (a,b) => a + b;  
val f2 = fn a => fn b => a + b;  
  
- f1 (5, 2);  
val it = 7 : int  
- f2 5 2;  
val it = 7 : int
```

Currying

Técnica que permite "carregar" argumentos ao longo da cadeia de funções retornadas a partir de uma chamada.

Ideia: Função recebe valor para argumento a_1 , e retorna uma nova função com a_1 substituído por seu valor. Esse processo é repetido para cada argumento a_n até que todos os argumentos sejam substituídos.

Exemplo: Soma

```
- val soma = fn a => fn b => a + b;
val soma = fn : int -> int -> int
- soma 2;
val it = fn : int -> int (* equivalente a fn b => 2 + b *)
- soma 2 4;
val it = 6 : int
```

Exemplo: Exponenciação

```
- fun pow b e = if e > 1 then b * pow b (e-1) else b;
val pow = fn : int -> int -> int
- pow 2;
val it = fn : int -> int (*fn e => if e > 1 then 2 * pow 2 (e-1) else 2;*)
- pow 2 3;
val it = 8 : int
```

Currying: Vantagens

1. Não é necessário mais trabalhar com tuplas para simular múltiplos parâmetros
 2. **Possibilidade de computar as funções parcialmente**
-

Exemplos Básicos

```
- val somaQuatro = soma 4;
val somaQuatro = fn : int -> int
- somaQuatro 2;
val it = 6 : int
```

```
fun square b = pow b 2;
val cube = fn b => pow b 3;

- square 3;
val it = 9 : int
- cube 2;
val it = 8 : int
```

Exemplo Completo: Merge Sort

```
fun mergeSort L comp = let
  fun dividir nil = (nil, nil)
  | dividir [a] = ([a], nil)
  | dividir (h1::h2::t) = let
      val (esq, dir) = dividir t
    in
      (h1::esq, h2::dir)
    end;

  fun juntar(nil, nil) = nil
  | juntar (a, nil) = a
  | juntar (nil, b) = b
  | juntar (a::ta, b::tb) = if comp(a, b)
      then a::juntar(ta, b::tb)
      else b::juntar(a::ta, tb);

  fun ordenar nil = nil
  | ordenar [a] = [a]
  | ordenar L = let
      val (esq, dir) = dividir L
      val esqSorted = ordenar esq
      val dirSorted = ordenar dir
    in
      juntar(esqSorted, dirSorted)
    end;
in
  ordenar L
end;

fun ordenaCrescente L = mergeSort L (op <);
fun ordenaDecrescente L = mergeSort L (op >);

- ordenaCrescente [100,1,40,~2,~25,14,4,234,6,10,0];
val it = [~25,~2,0,1,4,6,10,14,40,100,234] : int list
- ordenaDecrescente [100,1,40,~2,~25,14,4,234,6,10,0];
val it = [234,100,40,14,10,6,4,1,0,~2,~25] : int list
```

Funções Pré-definidas

Várias linguagens disponibilizam funções de alta ordem pré-definidas em suas APIs.

1. `map`
2. `foldr`

3. foldl

map

- Aplica uma função a cada elemento da lista.
- Retorna uma lista de mesmo tamanho, transformada pela função.
- Parâmetros: (a) função de transformação, (b) lista

```
('a -> 'b) -> 'a list -> 'b list
```

```
- map ~ [1,2,3,4,5];  
val it = [~1,~2,~3,~4,~5] : int list
```

```
- map (fn x => x mod 2 = 0) [1,2,3,4,5,6,7,8];  
val it = [false,true,false,true,false,true,false,true] : bool list
```

foldr & foldl

- Combina os elementos da lista em um único valor.
- Lógica de combinação é definida pela função.
- Parâmetros: (a) função de combinação, (b) valor inicial, (c) lista

```
('a * 'b -> 'b) -> 'b -> 'a list -> 'b
```

foldr

Processamento começa do elemento mais a direita

```
val it = fn : ('a * 'b -> 'b) -> 'b -> 'a list -> 'b  
- foldr (op *) 1 [1,2,3,4,5];  
val it = 120 : int  
- foldr (op ^) "" ["o", "Rato", "Roeu"];  
val it = "oRatoRoeu" : string
```

```
- val somatorio = foldr (op +) 0;  
val somatorio = fn : int list -> int
```

```
- somatorio [1,2,3,4,5];  
val it = 15 : int
```

foldl

Processamento começa do elemento mais a esquerda

```
- foldl (op ^) "" ["o", "Rato", "Roeu"];  
val it = "RoeuRatoo" : string
```

```
- fun reverso L = foldl (fn (a,b) => a::b) [] L;  
val reverso = fn : 'a list -> 'a list  
- reverso [1,2,3,4,5,6];  
val it = [6,5,4,3,2,1] : int list
```

Conclusão

- Trabalhar com funções como valores aumenta consideravelmente a expressividade de uma linguagem de programação.
- Esse recurso permite programar com sintaxe muito próxima ao adotado em cálculo lambda.
- Técnicas como funções anônimas, currying, e funções de alta ordem são possíveis graças a essa forma de manipulação.